

Правительство Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего профессионального образования
«Национальный исследовательский университет»
«Высшая школа экономики»
Нижегородский кампус

Факультет математики, информатики и компьютерных наук
Кафедра фундаментальной математики

КУРСОВАЯ РАБОТА
РЕАЛИЗАЦИЯ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ
ОПТИМИЗИРОВАННЫХ АЛГОРИТМОВ КРАСКАЛЛА И
ПРИМА

Выполнил:

Студент 4 курса группы 22 ФМ
Турсунов Данил Вячеславович

Научный руководитель:

Малышев Дмитрий Сергеевич

Нижний Новгород
Май 2026 г.

Содержание

1	Введение	3
1.1	Теоретическая часть	3
1.2	Алгоритмы решения задачи	4
2	Алгоритмическая часть	5
2.1	Описание	5
2.2	Алгоритм Краскалла	5
2.3	Оптимизация алгоритма Краскалла	6
2.4	Пример работы алгоритма Краскалла	7
2.5	Алгоритм Прима	7
2.6	Оптимизация алгоритма Прима	8
2.7	Двоичная куча	9
2.8	Фибоначчиева куча	10
2.9	Система непересекающихся множеств	10
2.10	Пример работы алгоритма Прима	11
3	Вычислительный эксперимент	12
3.1	Генерация данных	12
3.2	Сравнительный анализ	13
4	Заключение	14
4.1	Рассмотренные задачи	14
4.2	Достигнутые результаты	14
4.3	Ссылка на GitHub	14
5	Код графической части	15
5.1	Проверка оценки сложности работы алгоритмов	15
5.2	Сравнение абсолютного времени выполнения алгоритмов	16
6	Код алгоритмической части	18
6.1	Алгоритм Краскалла	18
6.2	Система непересекающихся множеств	18
6.3	Алгоритм Прима	19
6.4	Очередь с приоритетами	20
6.5	Генерация графов	21
7	Список литературы	23

1 Введение

1.1 Теоретическая часть

В данной курсовой работе будет рассматриваться задача построения минимального остовного дерева по заданному связному неориентированному взвешенному графу.

Перед описанием непосредственно рассматриваемой задачи и алгоритмов, с помощью которых эта задача решается, дадим определения основным понятиям, фигурирующим в курсовой работе:

Определение 1. (Конечный граф)

Конечным графом называется упорядоченная пара (V, E) , для которой выполнены следующие условия:

- 1) V - непустое конечное множество вершин;
- 2) E - множество пар вершин, называемых рёбрами.

Определение 2. (Неориентированный граф)

Неориентированный граф - граф, в котором рёбра не имеют направления, то есть E состоит из неупорядоченных пар $\{u, v\}$.

Определение 3. (Взвешенный граф)

Взвешенный граф - граф, в котором каждому ребру присвоен числовой вес.

Определение 4. (Инцидентность)

Если граф содержит ребро $e = (a, b)$, то каждую из вершин a, b называют инцидентной ребру e и говорят, что вершины a и b соединены ребром e .

Определение 5. (Путь в графе)

Путём в графе называют конечную последовательность его вершин и рёбер вида: $b_0, (b_0, b_1), b_1, \dots, b_{i-1}$,
1. Число k называется длиной пути, оно совпадает с числом входящих в него рёбер.

Определение 6. (Связность графа)

Граф называют связным, если любые две его вершины можно соединить путём.

Определение 7. (Цикл в графе)

Циклом длины $k \in \mathbb{N}$ в графе называют конечное подмножество его вершин и рёбер вида $\{b_0, (b_0, b_1), b_1, \dots, b_{i-1}, (b_{i-1}, b_i), b_i, \dots, b_{k-1}, (b_{k-1}, b_0)\}$. Простым циклом называют цикл, у которого все вершины и рёбра попарно различны.

Определение 8. (Дерево)

Дерево — связный неориентированный граф без циклов.

Определение 9. (Остовное дерево)

Для связного неориентированного графа $G = (V, E)$ остовное дерево $T = (V, E')$ — это подграф, удовлетворяющий условиям:

- 1) T является деревом,

- 2) $V(T) = V(G)$ (содержит все вершины исходного графа),
- 3) $E' \subseteq E$ (рёбра принадлежат исходному графу).

Определение 10. (Минимальное остовное дерево)

Для взвешенного связного неориентированного графа G минимальное остовное дерево — это остовное дерево $T \subseteq G$, такое что сумма весов рёбер T минимальна среди всех возможных остовных деревьев,

В дальнейшем под словом “граф” будем понимать связный неориентированный взвешенный граф.

1.2 Алгоритмы решения задачи

Алгоритм поиска минимального остовного дерева насчитывает практически вековую историю: в 1920-х годах правительство Чехословакии столкнулось с проблемой электрификации сельской местности, требовалось соединить все деревни электросетью, потратив минимальное количество средств. Здесь и появляется аналогия с вершинами и рёбрами: вершины - деревни, рёбра - стоимость прокладки кабеля.

И в 1926 году чешский математик Отто Боровка, решая этот вопрос, впервые описал алгоритм поиска минимального остовного дерева, который впоследствии стал называться алгоритмом Боровки. Однако его работа не получила широкого распространения, и только в 1950-х годах, с активным развитием теории алгоритмов, учёные Прим и Краскалл разработали собственные алгоритмы нахождения минимальных остовных деревьев. Эти алгоритмы и будут рассмотрены в курсовой работе.

В 1956 году Джозеф Краскалл опубликовал собственный алгоритм нахождения минимального остовного дерева. Основная идея алгоритма заключается в использовании системы непесекающихся множеств для проверки связности текущего дерева и, в случае несвязности, добавления минимального ребра к текущему дереву.

В 1957 году Роберт Прим независимо от него разработал собственный алгоритм, решая задачу прокладки телефонных кабелей. Основная идея алгоритма - пошаговое наращивание дерева за счёт минимальных рёбер, при этом минимальные рёбра хранятся в куче (приоритетная очередь). Примечательно, что алгоритм Прима практически полностью совпадает с алгоритмом Дейкстры, который решает задачу поиска минимального пути; единственное различие заключается в выборе ребра: в алгоритме Дейкстры ребро выбирается по суммарному пути.

2 Алгоритмическая часть

2.1 Описание

Курсовая работа представляет собой написание программы (кода), которая состоит из 2-х частей: алгоритмической и аналитической. Изначально запускается алгоритмическая часть кода, которая написана на языке программирования C++, в ней реализованы следующие подпрограммы:

- алгоритм Краскалла;
- оптимизация алгоритма Краскалла;
- алгоритм Прима;
- оптимизация алгоритма Прима;
- генератор графов;
- итоговая подпрограмма, которая генерирует графы различного количества вершин и рёбер и запускает на них алгоритмы Краскалла и Прима, замеряя время построения минимального остовного поддерева и выводя в отдельные файлы (для каждой пропорции количества рёбер и количества вершин создаётся отдельный файл) количество вершин и время построения.

Далее запускается аналитическая часть, состоящая из двух подпрограмм, каждая из которых берёт информацию из файлов, сгенерированных алгоритмической частью:

- программа, проводящая сравнительный анализ скорости работы алгоритмов Краскалла и Прима. Она строит отдельный график времени, потребовавшегося на построение минимального остовного дерева, в зависимости от количества вершин для каждой пропорции количества рёбер и количества вершин;
- программа, строящая графики отношения времени выполнения алгоритмов Краскалла и Прима к оценке их алгоритмической сложности.

В следующих подглавах подробно рассматривается каждая подпрограмма алгоритмической части отдельно, кроме генерации вершин, которая по смыслу больше подходит к аналитической части, поэтому генерация будет рассматриваться в отдельной главе, в которой будет обсуждаться именно аналитическая часть.

2.2 Алгоритм Краскалла

В алгоритме Краскалла минимальное остовное дерево строится поэтапно, путём добавления ребёр исходного графа к графу без рёбер, от минимального к максимальному, в случае, если это ребро соединяет вершины, принадлежащие разным компонентам связности строящегося графа.

Алгоритм Краскалла реализован в виде функции, которая принимает на вход граф, представленный в виде списка рёбер, и возвращает граф, представленный в виде списка смежности. Представления графа на входе и на выходе различаются в силу того, что изначально необходимо отсортировать рёбра по возрастанию, что делать по спискам смежности достаточно проблематично.

В написанной программе функция действует следующим образом: изначально создаётся пу-

стой граф (его список рёбер), который после выполнения алгоритма будет представлять из себя получившееся минимальное остовное дерево, а список рёбер изначального графа сортируется по возрастанию. Далее начинается непосредственно алгоритм Краскалла: перебираем все рёбра от меньшего к большему, далее, в случае, если очередное ребро соединяет рёбра, которые до сих пор не были соединены путём в строящемся графе, то это ребро добавляется в этот граф. После цикла в виде списка смежности возвращается получившийся граф. По окончании работы алгоритма будет построен граф, который будет являться минимальным остовным деревом изначального графа. Корректность алгоритма доказана в прикреплённых источниках.

2.3 Оптимизация алгоритма Краскалла

В описании алгоритма Краскалла был упущен момент, связанный в проверкой двух вершин на то, соединены ли вершины путём или нет. Для проверки этого факта используется структура данных, называемая системой непересекающихся множеств.

Система непересекающихся множеств представляет собой множество вершин, указывающих на своего предка. То есть, по сути, конкретное множество представляет собой иерархическую структуру, дерево, корнем которого является вершина-представитель множества. Это сделано для того, чтобы множества можно было объединить, указав у вершины-представителя предка (другими словами, подвесив вершину-представителя к другой), который является членом другого множества (необязательно представителем). Таким образом, для объединения множеств двух вершин необходимо:

- 1) найти вершины-представителей множеств этих двух вершин;
- 2) подвесить вершину-представителя одного множества к вершине-представителю другого множества.

В алгоритмической части программы СНМ реализована как класс DSU, у которого есть 2 метода: метод `find_parent` и метод `unite`, которые ищут вершину-представителя конкретной вершины и объединяют множества двух вершин (необязательно представителей) соответственно.

В методе `find_parent` на вход принимается вершина, далее идёт подъём по её предкам вплоть до того момента, когда у вершины не будет предка, а, значит, она является корнем дерева и вершиной-представителем. Соответственно, метод возвращает вершину-представителя.

Метод `unite` принимает на вход 2 вершины, находит у них вершины-представителей и подвешивает одну вершину представителя к другому.

Однако, с такой реализацией скорость объединения множеств может становиться линейной, превращая итоговую скорость работы алгоритма в квадратичную. Для более быстрого объединения будем пользоваться эвристическими подходами.

Первая эвристика заключается в подвешивании менее глубокого дерева к более глубокому. При таком подходе глубина дерева не увеличится (значит, и не увеличится скорость поиска вершины-представителя), кроме случая, когда глубина обоих деревьев одинакова. Для внедрения этой эвристики в программу придётся хранить глубину дерева.

Вторая эвристика заключается в том, чтобы обновлять предка у вершин, через которые

мы прошли при поиске вершины представителя. Будем запоминать, через какие вершины мы прошли и, когда дойдём до вершины представителя, подвесим все вершины к вершине-представителю, сокращая таким образом маршрут между вершиной и вершиной-представителем в будущем. При таком подходе глубина перестанет подсчитываться точно и станет оценкой глубины сверху.

После внедрения эвристик в программу, множества будут объединяться практически мгновенно, оценка сложности будет равняться $O(\alpha(n))$, где $\alpha(n)$ - обратная к функции Аккермана (настолько маленькая, что можно принять за константу).

В реализации алгоритма Краскалла перед циклом, перебирающим рёбра по возрастанию, создаётся структура СНМ, размер которой равен количеству вершин исходного графа. Изначально все вершины принадлежат разным множествам. Далее, в цикле, в случае добавления ребра множества, соответствующие инцидентным этому ребру вершинам, объединяются в одно множество.

2.4 Пример работы алгоритма Краскалла

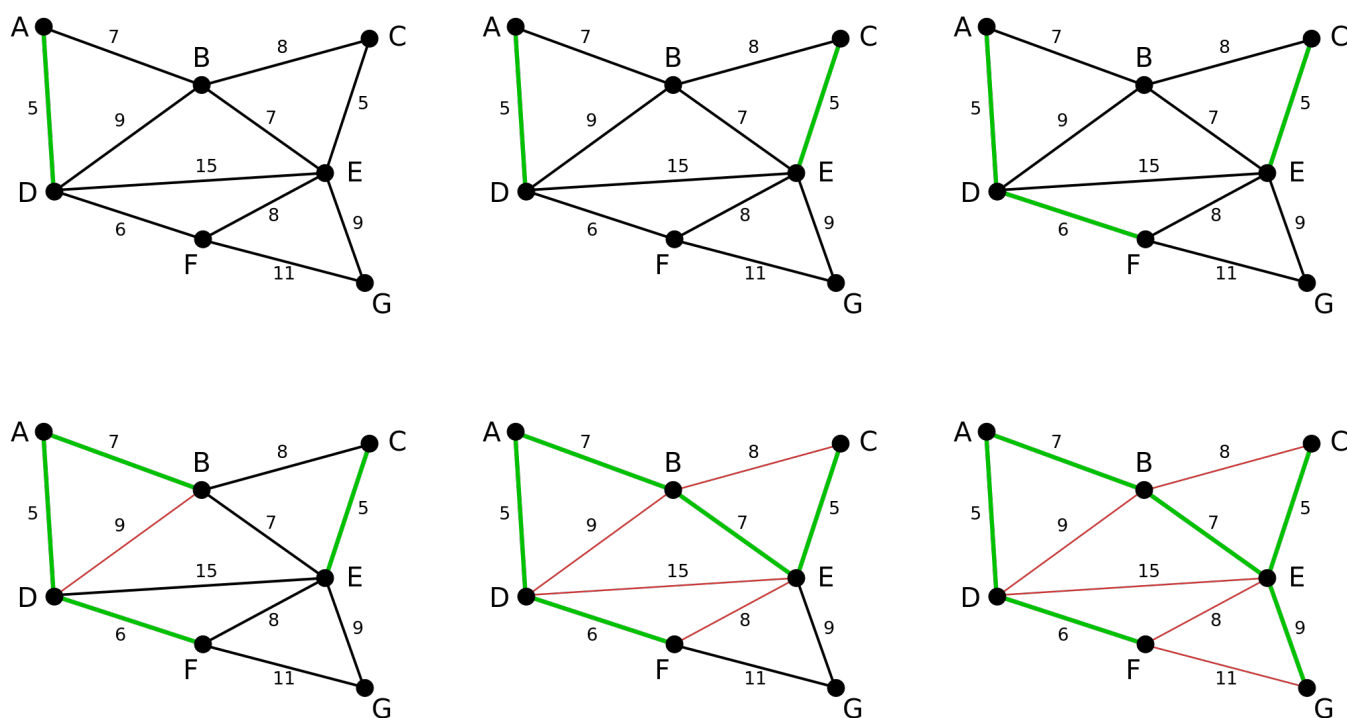


Рис. 1: Пример работы алгоритма Краскалла (6 шагов)

2.5 Алгоритм Прима

В алгоритме Прима минимальное остовное дерево строится жадно, путём поэтапного добавления рёбер к строящемуся дереву. Очередное добавленное ребро выбирается минимальным из тех, кто соединяет дерево с вершинами, в дерево ещё не добавленные.

Алгоритм Прима реализован в виде функции, которая принимает на вход список смежности изначального графа, возвращает тоже список смежности.

В программе функция действует следующим образом: инициализируется пустой список смежности, соответствующий строящемуся минимальному остовному дереву, список, хранящий предка каждой вершины, а также список, где у каждой вершины стоит отметка, добавлена она в дерево или ещё нет. Далее запускается цикл, у которого будет столько итераций, сколько вышло вершин в исходном графе.

В отдельно взятой итерации происходит следующее: находится вершина, которую на этой итерации добавим в дерево, то есть такую вершину, которая в дерево ещё не добавлена, а ребро, соединяющее её с деревом, является минимальным среди всех таких рёбер.

Далее запускается вложенный цикл, где со всех соседей добавленной вершины собирается информация: у них обновляется предок и запоминается дистанция до этих вершин через только что добавленную. После агрегации информации со всех соседей добавленной вершины вложенный цикл заканчивается, а внешний цикл переходит в очередную итерацию.

Далее, по окончании работы внешнего цикла, то есть, когда для всех вершин был добавлен предок (то есть, по сути, мы добавляли раньше не вершину в непосредственно дерево, а запоминали её предка в этом дереве и вес ребра, который соответствует ребру между вершиной и предком), запускается цикл обхода списка с предками и расстоянием до предков. В этом цикле как раз и происходит постройка минимального остовного дерева по информации, полученной из алгоритма выше. После работы цикла функция возвращает построенное минимальное остовное дерево. Корректность работы алгоритма доказана в прикреплённых источниках.

2.6 Оптимизация алгоритма Прима

В главе выше умышленно был упущен момент выбора очередной вершины, которую планируется добавить в строящееся дерево.

Тривиальный алгоритм этой процедуры состоит в том, чтобы циклом пройти по всем ещё не добавленным вершинам и посмотреть все их рёбра, найти среди них то ребро, что соединяет текущую вершину с деревом, и является минимальным среди таких рёбер, то есть просто запомнить минимальное ребро.

Очевидно, что этот алгоритм в худшем случае пройдёт по всем вершинам и по всем рёбрам, что превращает алгоритм Прима в квадратичный относительно количества вершин и рёбер, поэтому в реализованном алгоритме используется оптимизация, основанная на очереди с приоритетами.

Очередь с приоритетами - структура данных, которая является бинарным деревом с одним свойством - значения в вершинах меньше, чем значения в потомках (или больше, чем значения предков). С таким свойством очередь с приоритетами отождествляется с кучей минимумов, в дальнейшем будем использовать именно такой термин.

Куча минимумов позволяет находить минимальное значение практически моментально, в нашем случае, будем находить минимальное ребро, удовлетворяющее условиям. В программе куча минимумов реализована как класс, обладающий полями и методами. Реализована очередь с приоритетами в программе не как дерево, а как список, где поиск левого или правого ребёнка, а также предка, осуществляется по индексам $2i + 1$, $2i + 2$, $(i - 1)/2$, где i - индекс текущей

вершины соответственно.

Для кучи минимумов реализованы следующие методы: добавление элемента (он не более чем за $O(\log n)$ займёт нужное место в структуре данных), удаление элемента (тоже занимает $O(\log n)$), а также метод, позволяющий узнать минимальный элемент в куче за $O(1)$. Помимо этого реализованы методы, благодаря которым работают перечисленные выше методы, являющиеся публичными в смысле объектно-ориентированного программирования.

Вернёмся к алгоритму Прима. В цикле, где происходит поиск очередной вершины, вызывается функция `find_min`, которая возвращает подходящую вершину. Эта функция принимает на вход кучу минимумов, состоящую из соседних строящемуся дереву рёбер, и берёт рёбра из этой кучи одно за другим до тех пор, пока не найдёт ребро, соединяющее ещё не использованную вершину с деревом, это ребро и будет являться минимальным среди таких.

При таком подходе итоговая сложность работы алгоритма Прима будет $O((n + m)\log n)$.

2.7 Двоичная куча

Для поиска ребра минимального веса требуется структура данных, которая умеет быстро добавлять в себя новые рёбра, удалять использованные рёбра и находить среди рёбер минимальное.

Для быстрого нахождения минимального элемента существует структура данных, называемая кучей. Кучу можно реализовать несколькими способами, обладающими различным временем работы. В этой главе рассматривается двоичная куча.

Введём на множестве рёбер отношение порядка так, что ребро с меньшим весом меньше, чем ребро с большим весом.

Двоичная куча представляет из себя иерархическую структуру, дерево, в котором корнем является самый маленький элемент, а все вершины хранят номер предка. Для каждой вершины дерева выполняется правило: у неё может быть не более 2 вершин, считающих её предком (другими словами, не более 2 детей), кроме того, вершина должна быть меньше, чем её дети. В нашем случае вершинами в куче являются рёбра, и корневой вершиной будет являться ребро с минимальным весом.

Двоичная куча реализована в алгоритмической части как класс `BinaryHeap`. Само дерево кучи хранится как вектор `heap`, хранящий рёбра. В векторе поддерживается следующий инвариант: корень дерева хранится под индексом 0, а дети вершины под индексом i хранятся под индексами $2i + 1$ и $2i + 2$. Тогда родитель вершины с индексом i будет храниться под индексом $(i - 1)/2$, деление целочисленное.

Изначально вектор `heap` пуст, потому что в кучу ещё ничего не добавлено. Добавление в двоичную кучу происходит следующим образом: к уже имеющемуся вектору `heap` в конце добавляется элемент. Далее, этот элемент по формуле выше находит своего предка и сравнивает себя с ним. Если элемент меньше своего предка, то они меняются местами в векторе `heap`. Данное действие повторяется для добавленной вершины до тех пор, пока предок не окажется меньше или пока не окажется, что добавленная вершина стала корневой, то есть у неё индекс 0. Данный алгоритм называется просеиванием вверх и реализуется в методе `push` класса

BinaryHeap. Данная операция имеет сложность $O(\log E)$.

После взятия наименьшего элемента из кучи его следует удалить, потому что он уже использован, и хранить его не имеет смысла. Взятие элемента происходит тривиально, а удаление элемента происходит следующим образом: самый последний элемент кучи встаёт на место самого маленького элемента, то есть занимает индекс 0. Далее этот элемент меняется местами с наименьшим из своих детей, если ребёнок меньше данной вершины. Это действие повторяется для данной вершины до тех пор, пока у вершины существует ребёнок, меньший её самой. Данный алгоритм называется просеиванием вниз и реализуется в методе `pop` класса **BinaryHeap**. Данная операция имеет сложность $O(\log E)$.

Для внедрения данной структуры данных в алгоритм Прима реализована отдельная стратегия в классе **BinaryHeapStrategy**, которая содержит в себе поля `dist` и `used`. Первое является экземпляром класса двоичной кучи, второе является вектором, хранящим 1 по индексу, соответствующему номеру вершины, если эта вершина уже добавлена в минимальное остовное дерево, и 0 в противном случае. Добавление в двоичную кучу происходит при помощи метода `update`. Ключевым является метод `find_min`, который берёт минимальный элемент из кучи, и, если он не является использованным, возвращает его. Если его уже использовали, то удаляем этот элемент и берём следующий.

Теоретическая сложность алгоритма Прима с использованием двоичной кучи составляет $O((E + V) \log V)$, что показывается в [КОРМЕН ВСТАВИТЬ НОМЕР].

Пример работы двоичной кучи приведён на рисунке (СДЕЛАТЬ РИСУНОК).

2.8 Фибоначчиева куча

бла бла бла

2.9 Система непересекающихся множеств

Перед описанием системы непересекающихся множеств вспомним основную идею алгоритма Краскала: после сортировки всех рёбер исходного графа по весу начинается перебор рёбер от меньшего к большему. В случае, если ребро соединяет вершины, которые в новом графе принадлежат различным компонентам связности, ребро добавляется в новый граф. В противном случае ребро не добавляется, происходит переход к следующему ребру.

Ключевым моментом является проверка на то, соединяет ли ребро вершины из разных компонент связности. Наивное решение этой задачи для каждой такой проверки запускало бы поиск в глубину, алгоритмическая сложность такого решения составляет $O(V + E)$, что привело бы к общей теоретической сложности алгоритма Краскала $O(VE + E^2)$.

Для более оптимального решения этой задачи требуется структура данных, которая могла бы достаточно быстро проверять, лежат ли вершины в одной компоненте связности, и делать из двух конкретных компонент связности одну после добавления ребра. Имеет смысл решать более общую задачу, рассматривая не компоненты связности, а множества.

Примером такой структуры данных может являться система непересекающихся множеств

(СНМ). Она представляет из себя структуру данных, в которой хранится вектор предков вершин. Индексами этого вектора являются номера вершин. Вершина, считающая предком себя, называется представителем множества. Изначально каждая вершина является представителем множества, потому что новый граф представляет из себя набор вершин без рёбер. В программе СНМ реализована как класс DSU (disjoint-set union), вектор предков имеет название `parent`.

Система непересекающихся множеств является иерархической структурой, где каждое множество является деревом. Поэтому, чтобы объединить 2 множества вершин x и y , необходимо сделать следующее: 1) найти представителей множеств вершин x и y ; 2) у одного из представителей указать предком другого представителя, другими словами, подвесить одну вершину к другой, что поддерживает иерархическую структуру.

Поиск представителей происходит рекурсивно: найти представителя вершины x — это представитель вершины-предка x . Если вершина считает предком саму себя, то она является представителем множества. В коде данную задачу выполняет метод `find_parent`, который принимает на вход номер вершины x .

Поиск представителей можно оптимизировать, используя эвристику спрямления путей. Она заключается в том, чтобы на обратном пути из рекурсии всем промежуточным вершинам обновить предка на представителя. Тогда при последующих прохождениях не придётся проходить через промежуточные вершины, через которые мы проходили раньше, что существенно ускоряет время работы программы.

Подвешивание одной вершины к другой реализуется тривиально, однако нетривиальным остаётся способ выбора той вершины, к которой будут подвешивать другую. Для этого в структуре данных содержится вектор, который хранит в себе размеры множества. Стоит отметить, что корректные размеры множества хранятся только у вершин-представителей, потому что другие вершины подвешиваться больше не будут. Теперь будем подвешивать другую вершину к той, у которой больше размер множества. В противном случае могут возникать высокие деревья, что делает поиск представителя достаточно долгим. В программе данную задачу выполняет метод `unite`, который принимает на вход 2 номера вершин x и y . Вектор размеров имеет название `size`.

Итоговая теоретическая сложность объединения двух множеств будет равняться $O(\alpha(V))$, где $\alpha(V)$ — обратная к функции Аккермана. Эта функция растёт очень медленно, поэтому при оценке общей скорости работы её принимают за константу. После внедрения СНМ в работу алгоритма, общая оценка сложности работы алгоритма Краскала составит $O(E \log E)$.

В реализации алгоритма Краскала СНМ используется следующим образом: после сортировки всех рёбер по возрастанию создаётся экземпляр класса DSU, названный `dsu`. Далее, при проходе цикла по всем рёбрам, пусть у рассматривается ребро `edge(from, to, cost)`. Если `dsu.find_parent(from)` не равен `dsu.find_parent(to)`, то вызывается `dsu.unite(from, to)`. После окончания цикла все множества объединяются в одно, а функция, реализующая алгоритм Краскала, возвращает получившийся граф.

Пример работы системы непересекающихся множеств приведён на рисунке (СДЕЛАТЬ РИСУНОК).

2.10 Пример работы алгоритма Прима

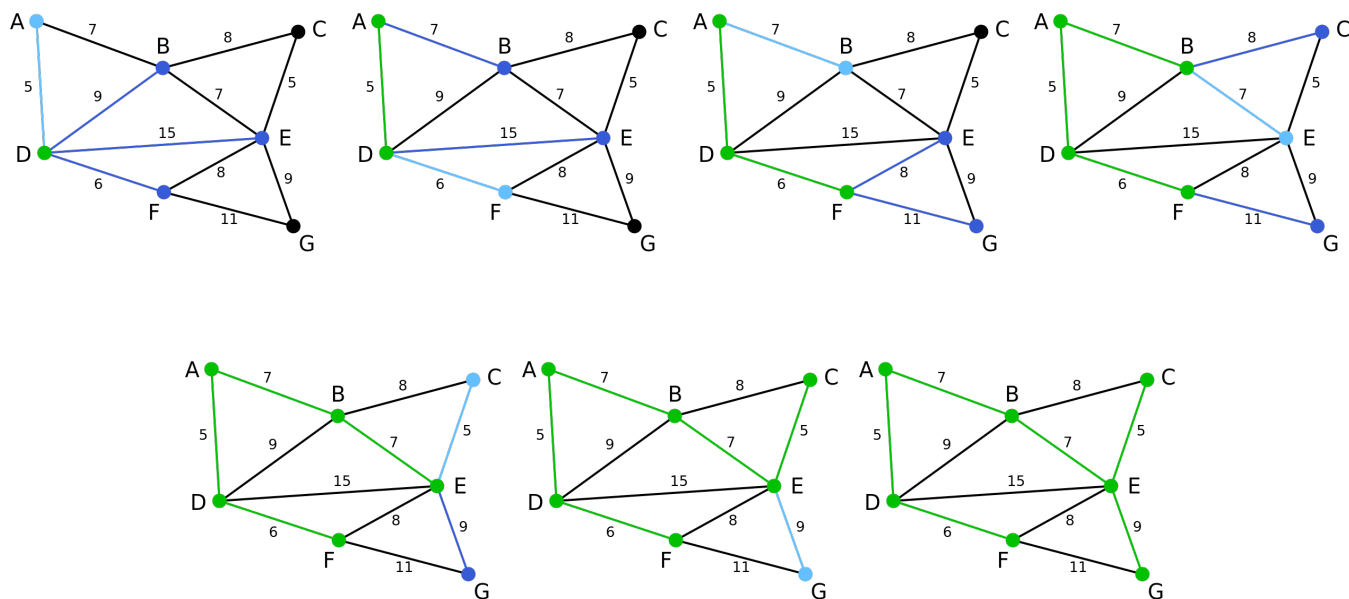


Рис. 2: Пример работы алгоритма Прима (7 шагов)

3 Вычислительный эксперимент

3.1 Генерация данных

Для сравнительного анализа двух алгоритмов необходимо измерять скорость работы алгоритма Прима и алгоритма Краскалла на графах, обладающих различными характеристиками: количество вершин, количество рёбер, то, насколько граф близок к полному.

Задача состоит в том, чтобы генерировать граф со случайными весами рёбер по заданному количеству вершин и рёбер (при этом подразумевается, что количество вершин и рёбер задано корректно, то есть можно построить связный граф с заданными характеристиками). Эта задача реализована в методе `fill` для списка рёбер, который реализован в программе как отдельный класс. Сначала создаётся экземпляр класса с заданными характеристиками, после чего при помощи вызова метода `fill` генерируются все рёбра корректным образом.

Генерация происходит следующим образом: все вершины соединяются рёбрами, образуя круг. Далее случайно выбирается 2 вершины, и, если нет рёбер, соединяющих эти 2 вершины, то ребро добавляется. Если же ребро уже есть, то ещё раз выбираем случайно 2 вершины и проделываем ту же процедуру, и так до тех пор, пока ребро не будет добавлено. Этот алгоритм повторяется, пока не будет добавлено необходимое количество рёбер.

В процедуре, описанной выше, вес ребра каждый раз случайно выбирается из равномерного распределения от 0 до 1.

В основной части программы, написанной на языке программирования C++, то есть алгоритмической части программы, происходит генерация графов 3-х типов при помощи метода `fill`. Типы графов выбраны следующим образом:

- 1) $E = 2V$
- 2) $E = V\sqrt{V}$
- 3) $E = V^2 / 4$

где E - количество рёбер в графе, а V - количество вершин в графе.

Для каждого типа количество вершин в сгенерированных графах проходит от 10 до 3000 с шагом в 50 вершин.

Для каждого отдельно взятого сгенерированного графа сохраняются в отдельный файл его характеристики, а также скорости работы алгоритмов Прима и Краскалла.

3.2 Сравнительный анализ

Для сравнительного анализа алгоритмов Прима и Краскалла было написано 2 скрипта на языке программирования Python.

Первый скрипт отвечает за сравнение скорости работы алгоритмов Прима и Краскалла между собой.

Скрипт считывает данные из файлов, содержащих информацию из предыдущего пункта, и, используя библиотеку `matplotlib`, строит графики зависимости времени выполнения программы от количества вершин для каждого из типов.

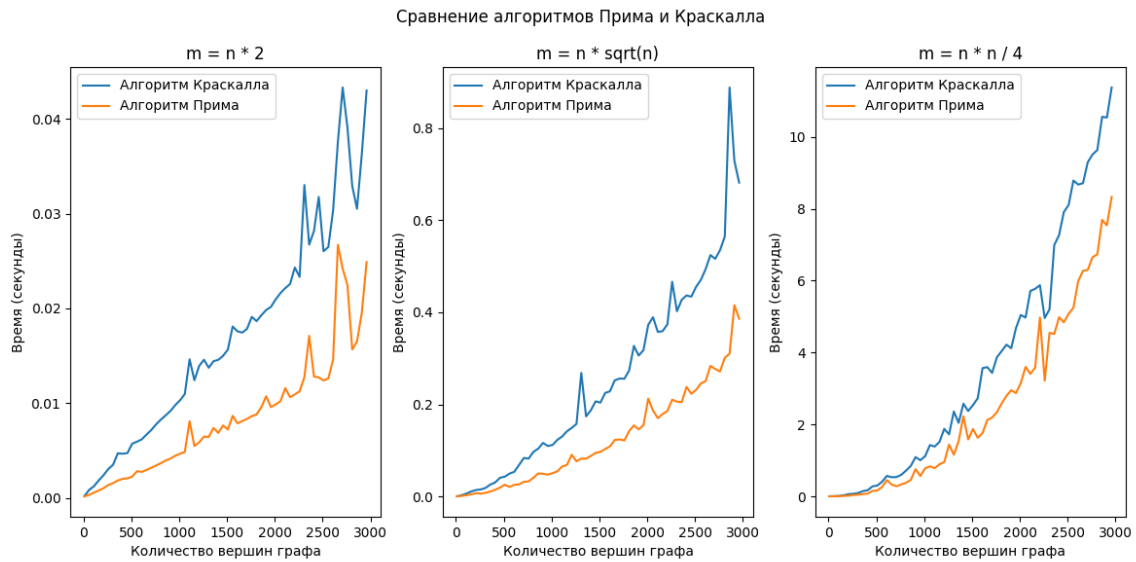


Рис. 3: Сравнение алгоритмов Прима и Краскалла

Из графиков можно видеть, что алгоритм Прима работает практически в 2 раза быстрее, чем алгоритм Краскалла, и на достаточно разреженных графах, и на графах, близких к полным.

Второй скрипт отвечает за проверку соответствия асимптотической сложности алгоритма реальным показателям.

Скрипт считывает данные из файлов, полученных выше, и, используя библиотеку `matplotlib`, строит графики зависимости отношения к оценке сложности от количества вершин графа для каждого из алгоритмов.

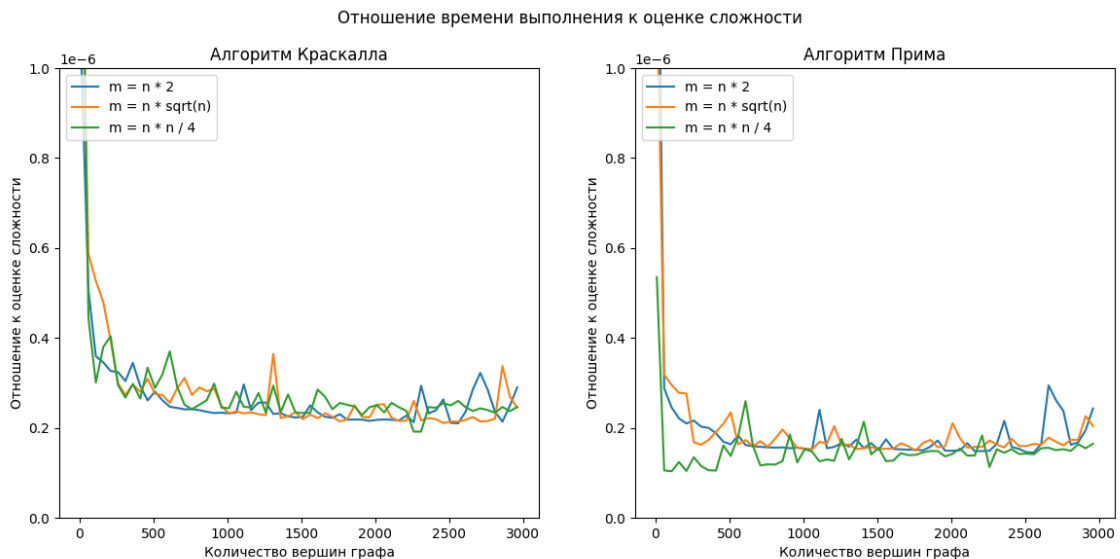


Рис. 4: Отношение времени выполнения к оценке сложности

Из графиков видно, что графики всех типов графов для каждого из алгоритмов лежат примерно на одной прямой, что говорит о том, что оценка сложности алгоритмов является правильной.

4 Заключение

4.1 Рассмотренные задачи

Изначально требовалось рассмотреть в курсовой работе следующие задачи:

- изучить алгоритмы Краскалла и Прима;
- реализовать алгоритмы Краскалла и Прима;
- провести сравнительный анализ алгоритмов Краскалла и Прима;

Однако круг задач постепенно расширялся, что привело к получению дополнительных результатов.

4.2 Достигнутые результаты

В курсовой работе, с учётом и поставленных изначально задач, и добавленных впоследствии, были достигнуты следующие результаты:

- алгоритмы Краскалла и Прима были написаны на языке программирования C++;
- были написаны оптимизации алгоритмов Краскалла и Прима;
- была написана программа на языке программирования Python, отрисовывающая графики скорости выполнения алгоритмов Краскалла и Прима;
- был проведён сравнительный алгоритмов Краскалла и Прима, в том числе и сравнительный анализ скорости их работы;

4.3 Ссылка на GitHub

Реализация алгоритмов Краскалла и Прима на языке программирования C++ и код, необходимый для их сравнительного анализа, на языке программирования Python находятся в открытом репозитории на GitHub по следующей ссылке: github.com/dan1lka257/coursework_3

5 Код графической части

5.1 Проверка оценки сложности работы алгоритмов

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 def count_edges_1_prim(vertex_count):
5     return (float(vertex_count) + 2 * float(vertex_count)) * np.log2(float(
6 vertex_count))
7
8 def count_edges_2_prim(vertex_count):
9     return (float(vertex_count) + np.sqrt(float(vertex_count)) * float(
10 vertex_count)) * np.log2(float(vertex_count))
11
12 def count_edges_3_prim(vertex_count):
13     return 2 * (float(vertex_count) + float(vertex_count) * float(vertex_count)
14 / 4) * np.log2(float(vertex_count))
15
16 def count_edges_1_kruskal(vertex_count):
17     return 2 * 2 * float(vertex_count) * np.log2(2 * float(vertex_count))
18
19 def count_edges_2_kruskal(vertex_count):
20     return np.sqrt(float(vertex_count)) * float(vertex_count) * np.log2(np.sqrt(
21 float(vertex_count)) * float(vertex_count))
22
23 def count_edges_3_kruskal(vertex_count):
24     return float(vertex_count) * float(vertex_count) / 4 * np.log2(float(
25 vertex_count) * float(vertex_count) / 4)
26
27 def read_data(filename, func):
28     x, y = [], []
29     with open(filename, 'r') as f:
30         for line in f:
31             parts = line.strip().split()
32             x.append(int(parts[0]))
33             y.append(float(parts[1]) / func(parts[0]))
34     return np.array(x), np.array(y)
35
36 x_kruskal_1, y_kruskal_1 = read_data('kruskalFile1.txt',
count_edges_1_kruskal)
x_kruskal_2, y_kruskal_2 = read_data('kruskalFile2.txt',
count_edges_2_kruskal)
x_kruskal_3, y_kruskal_3 = read_data('kruskalFile3.txt',
count_edges_3_kruskal)
x_prim_1, y_prim_1 = read_data('primFile1.txt', count_edges_1_prim)
x_prim_2, y_prim_2 = read_data('primFile2.txt', count_edges_2_prim)
x_prim_3, y_prim_3 = read_data('primFile3.txt', count_edges_3_prim)
```



```

37
38 plt.figure(figsize=(14, 6))
39
40 plt.subplot(1, 2, 1)
41 plt.plot(x_kruskal_1, y_kruskal_1, label='m = n * 2')
42 plt.plot(x_kruskal_2, y_kruskal_2, label='m = n * sqrt(n)')
43 plt.plot(x_kruskal_3, y_kruskal_3, label='m = n * n / 4')
44 plt.title('Kruskal')
45 plt.xlabel('The number of vertices in the graph')
46 plt.ylabel('Attitude to complexity assessment')
47 plt.ylim(0, 1e-6)
48 plt.legend(loc='upper left')
49
50 plt.subplot(1, 2, 2)
51 plt.plot(x_prim_1, y_prim_1, label='m = n * 2')
52 plt.plot(x_prim_2, y_prim_2, label='m = n * sqrt(n)')
53 plt.plot(x_prim_3, y_prim_3, label='m = n * n / 4')
54 plt.title('Prim')
55 plt.xlabel('The number of vertices in the graph')
56 plt.ylabel('Attitude to complexity assessment')
57 plt.ylim(0, 1e-6)
58 plt.legend(loc='upper left')
59
60 plt.suptitle('The ratio of execution time to complexity estimation')
61
62 plt.show()
63

```

5.2 Сравнение абсолютного времени выполнения алгоритмов

```

1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  def read_data(filename):
5      x, y = [], []
6      with open(filename, 'r') as f:
7          for line in f:
8              parts = line.strip().split()
9              x.append(int(parts[0]))
10             y.append(float(parts[1]))
11         return np.array(x), np.array(y)
12
13 x_kruskal_1, y_kruskal_1 = read_data('kruskalFile1.txt')
14 x_kruskal_2, y_kruskal_2 = read_data('kruskalFile2.txt')
15 x_kruskal_3, y_kruskal_3 = read_data('kruskalFile3.txt')
16 x_prim_1, y_prim_1 = read_data('primFile1.txt')
17 x_prim_2, y_prim_2 = read_data('primFile2.txt')
18 x_prim_3, y_prim_3 = read_data('primFile3.txt')

```

```

19
20 plt.figure(figsize=(14, 6))
21
22 plt.subplot(1, 3, 1)
23 plt.plot(x_kruskal_1, y_kruskal_1, label='Kruskal')
24 plt.plot(x_prim_1, y_prim_1, label='Prim')
25 plt.title('m = n * 2')
26 plt.xlabel('The number of vertices in the graph')
27 plt.ylabel('Time (seconds)')
28 plt.legend(loc='upper left')
29
30 plt.subplot(1, 3, 2)
31 plt.plot(x_kruskal_2, y_kruskal_2, label='Kruskal')
32 plt.plot(x_prim_2, y_prim_2, label='Prim')
33 plt.title('m = n * sqrt(n)')
34 plt.xlabel('The number of vertices in the graph')
35 plt.ylabel('Time (seconds)')
36 plt.legend(loc='upper left')
37
38 plt.subplot(1, 3, 3)
39 plt.plot(x_kruskal_3, y_kruskal_3, label='Kruskal')
40 plt.plot(x_prim_3, y_prim_3, label='Prim')
41 plt.title('m = n * n / 4')
42 plt.xlabel('The number of vertices in the graph')
43 plt.ylabel('Time (seconds)')
44 plt.legend(loc='upper left')
45
46 plt.suptitle('Comparison of the Prim and Kruskal algorithms')
47
48 plt.show()
49

```

6 Код алгоритмической части

6.1 Алгоритм Краскала

```
1  template <typename T, typename U>
2  bool comp(const Edge<T, U>& edge1, const Edge<T, U>& edge2) {
3      return edge1.cost < edge2.cost;
4  }
5
6  template <typename T, typename U>
7  Graph<T, U> Kruskal(Edges<T, U>& edges) {
8      Edges<T, U> new_edges(edges.vertexCount, 0);
9      std::sort(edges.edges.begin(), edges.edges.end(), comp<T, U>);
10     Dsu<U> dsu(edges.vertexCount + 1);
11     for (auto edge : edges.edges) {
12         if (dsu.find_parent(edge.from) != dsu.find_parent(edge.to)) {
13             dsu.unite(edge.from, edge.to);
14             new_edges.edges.emplace_back(edge);
15         }
16     }
17     Graph<T, U> graph(new_edges);
18     return graph;
19 }
20
```

6.2 Система непересекающихся множеств

```
1  template <typename U>
2  struct Dsu {
3      std::vector<U> parent;
4      std::vector<U> rank;
5      Dsu(U n) {
6          rank.resize(n, 1);
7          parent.resize(n, 0);
8          for (U i = 0; i < n; ++i) {
9              parent[i] = i;
10         }
11     }
12     U find_parent(U x) {
13         std::vector<U> way;
14         while (parent[x] != x) {
15             way.push_back(x);
16             x = parent[x];
17         }
18         for (auto i : way) {
19             parent[i] = x;
20         }
21         return x;
22     }
23 }
```

```

22     }
23     void unite(U x, U y) {
24         x = find_parent(x);
25         y = find_parent(y);
26         if (rank[x] == rank[y]) {
27             parent[x] = y;
28             ++rank[y];
29         } else if (rank[x] < rank[y]) {
30             parent[x] = y;
31         } else {
32             parent[y] = x;
33         }
34     }
35 };
36

```

6.3 Алгоритм Прима

```

1     template <typename T, typename U>
2     U find_min(BinaryHeap<std::pair<T, U>>& dist, std::vector<U>& used) {
3         while (used[dist.top().second]) {
4             dist.pop();
5         }
6         U min_dist_ind = dist.top().second;
7         used[min_dist_ind] = 1;
8         return min_dist_ind;
9     }
10
11     template <typename T, typename U>
12     Graph<T, U> Prim(Graph<T, U>& graph) {
13         Graph<T, U> new_graph(graph.vertexCount, 0);
14         std::vector<std::pair<U, T>> parent(graph.vertexCount + 1, std::pair<U, T>
15         >(-1, -1));
16         std::vector<U> used(graph.vertexCount + 1, 0);
17         BinaryHeap<std::pair<T, U>> dist;
18         dist.push({ 0, 1 });
19         for (U i = 0; i < static_cast<U>(graph.neig.size()); ++i) {
20             U min_dist_ind = 0;
21             try {
22                 min_dist_ind = find_min(dist, used);
23             }
24             catch (...) {}
25             for (U j = 0; j < static_cast<U>(graph.neig[min_dist_ind].size()); ++j) {
26                 if (!used[graph.neig[min_dist_ind][j].first]) {
27                     dist.push({ graph.neig[min_dist_ind][j].second, graph.neig[
28 min_dist_ind][j].first });
29                     parent[graph.neig[min_dist_ind][j].first].second = graph.neig[
30 min_dist_ind][j].second;
31                 }
32             }
33         }
34         return new_graph;
35     }
36

```

```

28         parent[graph.neig[min_dist_ind][j].first].first = min_dist_ind;
29     }
30 }
31 }
32 for (U i = 0; i < static_cast<U>(parent.size()); ++i) {
33     if (parent[i].first != -1) {
34         new_graph.neig[i].emplace_back(std::pair<U, T>(parent[i].first, parent[
i].second));
35         new_graph.neig[parent[i].first].emplace_back(std::pair<U, T>(i, parent[
i].second));
36     }
37 }
38 return new_graph;
39 }
40

```

6.4 Очередь с приоритетами

```

1  template <typename T, typename U = int64_t>
2  class BinaryHeap {
3      private:
4          std::vector<T> heap;
5
6          U parent(U i) const { return (i - 1) / 2; }
7          U left_child(U i) const { return 2 * i + 1; }
8          U right_child(U i) const { return 2 * i + 2; }
9
10         void sift_up(U i) {
11             while (i > 0 && heap[parent(i)] > heap[i]) {
12                 std::swap(heap[parent(i)], heap[i]);
13                 i = parent(i);
14             }
15         }
16
17         void sift_down(U i) {
18             U min_index = i;
19             U left = left_child(i);
20             U right = right_child(i);
21
22             if (left < static_cast<U>(heap.size()) && heap[left] < heap[min_index]) {
23                 min_index = left;
24             }
25             if (right < static_cast<U>(heap.size()) && heap[right] < heap[min_index])
26         {
27             min_index = right;
28         }
29         if (i != min_index) {
30             std::swap(heap[i], heap[min_index]);
31         }
32     }
33 };

```

```

30         sift_down(min_index);
31     }
32 }
33
34 public:
35 BinaryHeap() = default;
36
37 bool empty() const { return heap.empty(); }
38
39 const T& top() const {
40     if (empty()) {
41         throw std::out_of_range("Heap is empty");
42     }
43     return heap.front();
44 }
45
46 void push(const T& value) {
47     heap.push_back(value);
48     sift_up(heap.size() - 1);
49 }
50
51 void pop() {
52     if (empty()) {
53         throw std::out_of_range("Heap is empty");
54     }
55     heap[0] = heap.back();
56     heap.pop_back();
57     if (!empty()) {
58         sift_down(0);
59     }
60 }
61 };
62

```

6.5 Генерация графов

```

1 void fill() {
2     std::mt19937 gen(42);
3     std::uniform_int_distribution<> vertexGenerator(1, static_cast<int>(
vertexCount));
4     std::uniform_real_distribution<T> distGenerator(0.0, 1.0);
5
6     std::unordered_set<std::pair<U, U>> edges_uset;
7     for (U i = 1; i < vertexCount + 1; ++i) {
8         U new_from = i;
9         U new_to = i % vertexCount + 1;
10        if (new_from > new_to) {
11            std::swap(new_from, new_to);

```

```

12     }
13     T new_cost = distGenerator(gen);
14     edges.push_back(Edge<T, U>(new_from, new_to, new_cost));
15     edges_uset.insert(std::make_pair(new_from, new_to));
16 }
17 for (U i = 1; i < edgeCount - vertexCount + 1; ++i) {
18     U new_from = vertexGenerator(gen);
19     U new_to = vertexGenerator(gen);
20     if (new_from > new_to) {
21         std::swap(new_from, new_to);
22     }
23     T new_cost = distGenerator(gen);
24     Edge<T, U> new_edge = Edge<T, U>(new_from, new_to, new_cost);
25     if (new_from == new_to || edges_uset.count(std::pair<U, U>(new_from,
new_to))) {
26         --i;
27         continue;
28     }
29     edges.push_back(new_edge);
30     edges_uset.insert(std::pair<U, U>(new_from, new_to));
31 }
32 }
33

```

7 Список литературы

- [1] Borůvka O. *O jistém problému minimálním*. Práce Moravské Přírodovědecké Společnosti, 1926. Vol. 3, No. 3, pp. 37–58. URL: https://dml.cz/bitstream/handle/10338.dmlcz/500387/Boruvka_03-0000-49_1.pdf
- [2] Иванов А. В. *К истории двух знаменитых оптимизационных алгоритмов в теории графов*. История науки и техники, 2015. URL: <https://cyberleninka.ru/article/n/k-istorii-dvuh-znamenityh-optimizazionnyh-algoritmov-v-teorii-grafov>
- [3] Петров С. К. *Алгоритмы Прима и Крускала: сравнительный анализ*. Журнал вычислительной математики, 2018. URL: <https://elibrary.ru/item.asp?id=35121079>
- [4] Охотин А. *Лекция 2: Алгоритмы на графах*. 2024. URL: https://users.math-cs.spbu.ru/~okhotin/teaching/algorithms3_2024/okhotin_algorithms3_2024_12.pdf
- [5] Сидоров М. Н. *Применение алгоритмов Прима и Крускала в сетевых структурах*. Информационные технологии, 2020. URL: <https://elibrary.ru/item.asp?id=53807603>
- [6] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009. URL: [https://ducndc.github.io/assets/books/dsa/Introduction_to_Algorithms_Third_Edition_\(2009\).pdf](https://ducndc.github.io/assets/books/dsa/Introduction_to_Algorithms_Third_Edition_(2009).pdf)